ROYAL INSTITUTE
OF TECHNOLOGY

Bachelor's thesis

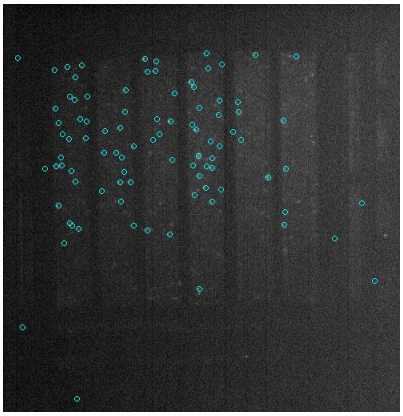
Blinking Quantum Dots: Statistics Through Image Processing

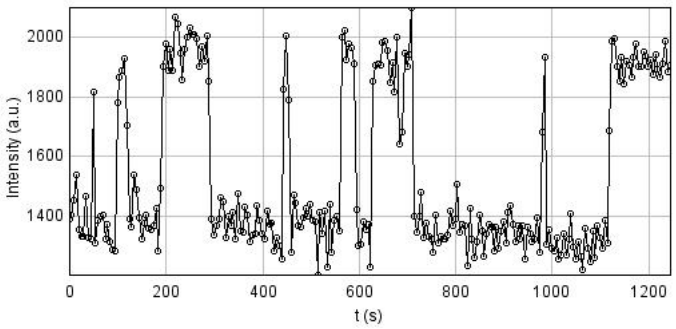
Authors:
Erik Gedeborg
830905-0139

Mikael Li
880819-1459

Examiner:
Jan Linnros

Supervisor:
Benjamin Bruhn





Material Physics department of ICT school, KTH

1 Abstract

In this project we have developed an image processing program called *Quantum Dot Analyzer* that is able to detect and record quantum dot blinking sequences. Our purpose with the program was to analyze blinking quantum dots and to generate statistics for further research. With the program we can record the size, blinking intensity over time for quantum dots and do statistics of on- and off-times.

We have used Java, ImageJ[1] and Matlab in the development process. Java is a widely used object-oriented programming language notable for its portability. ImageJ is a public domain image processing program based on Java. For generating statistics we used Matlab, which is a mathematical computing program. The Quantum Dot Analyzer was made to be used as a plugin for ImageJ.

The quantum dots show different characteristics – some blinks, some are constantly on, some are off for a long time and then suddenly flash a quick light. By plotting the on- and off-time (logarithmically) for the quantum dots, we could conclude that the quantum dots show very different behaviour.

[1] http://rsbweb.nih.gov/ij/. 2010-08-18

2 Table of Contents

Contents

1 Abstract **1**

2 Table of Contents **2**

3 Introduction **3**

4 Quantum Dot Analyzer **4**
4.1 Drift Correction 4
4.1.1 Adjust template ROIs 6
4.1.2 Correlation coefficient 7
4.2 Threshold 8
4.2.1 Set Mutliplier 8
4.2.2 Threshold map 9
4.2.3 Applying the thresholding 10
4.3 Extracting Quantum Dots 10
4.4 Plotting the distrubution for on- and off-time 12
4.5 Post-processing 12

5 Results **13**
5.1 Graphs of quantum dot light intensity 13
5.2 Log of data 13
5.3 Graphs of on- and off-time 13
5.4 Graph and histogram 14

6 Discussion **14**

7 Conclusion **15**

8 Recommendations **15**

9 References **17**

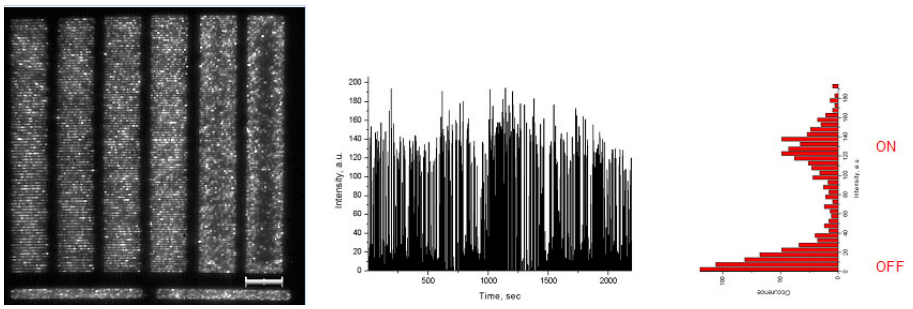
3 Introduction

Image processing is being used in many different applications. E.g. medical imaging can detect tumours in human bodies, face detection in cameras and photo manipulation (Adobe Photoshop) etc. In our case, we have been using image processing for analyzing quantum dots.

Quantum dots can be spherical, they are made of semiconductors and are typically 1-10 nm large. When optical pumping is applied, quantum dots show strong light emission, but still the quantum dots can come into a dark state where the light emission completely ceases, and then all of sudden become luminous with full intensity. This phenomenon is called "blinking", which might appear when electrons occupy a defect state which can be found at the interface to e.g. an oxide. This kind of luminescence includes very interesting physics which can teach us more about quantum dots and maybe later on use them in different applications.

At KTH's section of Material Physics, research has been made on individual silicon quantum dots which can be resolved optically in a microscope. This is because they are made with electron-beam-lithography and placed with a distance of at least one wavelength of light. During optical pumping with a laser with wavelength 325 nm, the quantum dots emit light in the range 600 – 800 nm which is being detected by a CCD-camera – see figure 1. The camera needs about 5 – 20 seconds of integration in order to be sure to detect the weak light that eminates from every quantum dot. By storing hundreds of consecutively taken pictures, we can notice that some quantum dots are blinking while some are constantly shining. We want to analyze this statistically!

Our task was to make a program that can load image sequences, apply a thresholding filter to the images, extract the blinking sequence from the detected quantum dots and generate statistics such as on- and off-time.



(a) (b) (c)

Figure 1: (a) CDD-camera picture with blinking quantum dots. (b) Diagram for intensity versus time for a quantum dot. (c) Histogram of intensities from (b).

4 Quantum Dot Analyzer

To solve the problem, it is useful to divide the analysis into parts concerning drift correction, thresholding, dot extraction and further analysis. The first three parts are mainly done in ImageJ whereas the last part is done mainly in MATLAB.

The program we have made is called Quantum Dot Analyzer. In this report we are going through the technical details.

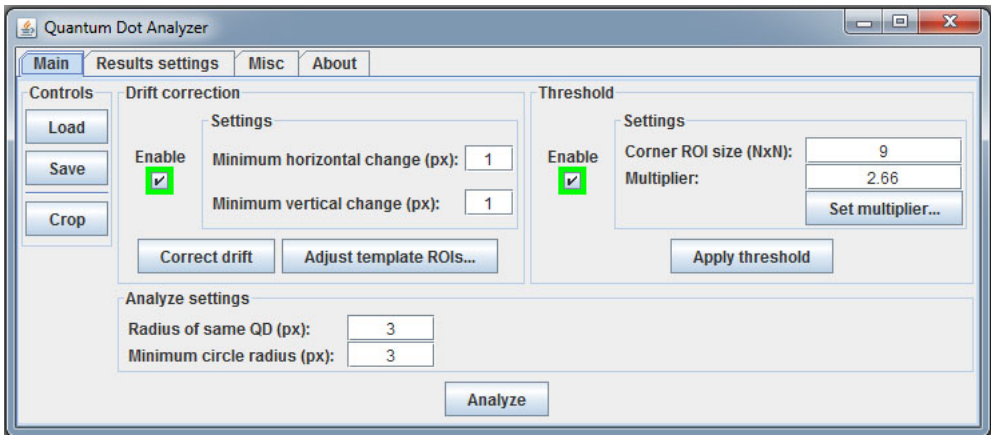


Figure 2: The Quantum Dot Analyzer.

4.1 Drift Correction

We have noticed that some image sequences exhibit lateral drift, meaning that from image to image the picture (due to sample or camera movement) is moving a little bit. This lateral drift is usually very small, for instance one or two pixels throughout the whole sequence, but for some image sequences for which the temperature of the camera setup has not been kept stable enough, the lateral drift may be very large. In either way, in order for the software to be able to identify a specific quantum dot throughout the series of images it is convenient to try to eliminate the lateral drift before the dot extraction part. The pseudo-code for a principal algorithm for drift correction is listed in algorithm 1.

Algorithm 1 Algorithm for drift correction

Require: Image stack I, reference/template image R
N = Number of images in the sequence
for $n = 1$ to $N - 1$ **do**
  for all displacements R' of the reference image **do**
    Compute the correlation coefficient of the displaced reference image R' and store into coefficient matrix/image C.
  end for
  Calculate the position $(x^{(n)}, y^{(n)})$ of the peak of the intensity in the resulting correlation map I_C.
  Determine the drift relative to the original image and store offset:
  $Dx[n]$ = x-offset
  $Dy[n]$ = y-offset
end for
Eliminate parts in D where there is unphysical/unreasonable fluctuation by for instance requiring that an increase/decrease in drift lasts a certain time.
Correct the drift
absMinOffsetX = |min (Dx)|, absMaxOffsetX = |max (Dx)|
absMinOffsetY = |min (Dy)|, absMaxOffsetY = |max (Dy)|
w = width(I) - (absMinOffsetX + absMaxOffsetX)
h = height(I) - (absMinOffsetY + absMaxOffsetY)
for $n = 1$ to $N - 1$ **do**
  if Dx[n] < 0 **then**
    x = |Dx(n)| + absMaxOffsetX
  else
    x = absMaxOffsetX - |Dx(n)|
  end if
  if Dy[n] < 0 **then**
    y = absMinOffsetY - |Dy(n)|
  else
    y = |Dx(n)| + absMaxOffsetY
  end if
  Crop I to region at (x, y) with size (w, h)
end for

The image width is W. First, the maximal offset is calculated in negative axis and positive axis. Every image has been assigned with a specific offset d, from that the program extracts the area which is common for all images. Relatively, each image gets the area's x-position, x, and width, w, is given by:

$$d < 0 : \begin{cases} x = |d| + |D^+| \\ w = W - (|D^-| + |D^+|) \end{cases}$$
$$d \geq 0 : \begin{cases} x = |D^+| - |d| \\ w = W - (|D^-| + |D^+|) \end{cases}$$

As the width common area is always the same, we get:

$$x = \begin{cases} |d| + |D^+|, d < 0 \\ |D^+| - |d|, d \geq 0 \end{cases}$$
$$w = W - (|D^-| + |D^+|)$$

Y-axis drifts is obtained in a similar way. All in all, we get:

$$x = \begin{cases} |d_x| + |D_x^+|, d_x < 0 \\ |D_x^+| - |d_x|, d_x \geq 0 \end{cases}$$
$$y = \begin{cases} |D_y^-| - |d_y|, d_y < 0 \\ |d_y| + |D_y^-|, d_y \geq 0 \end{cases}$$
$$w = W - (|D_x^-| + |D_x^+|)$$
$$h = H - (|D_y^-| + |D_y^+|)$$

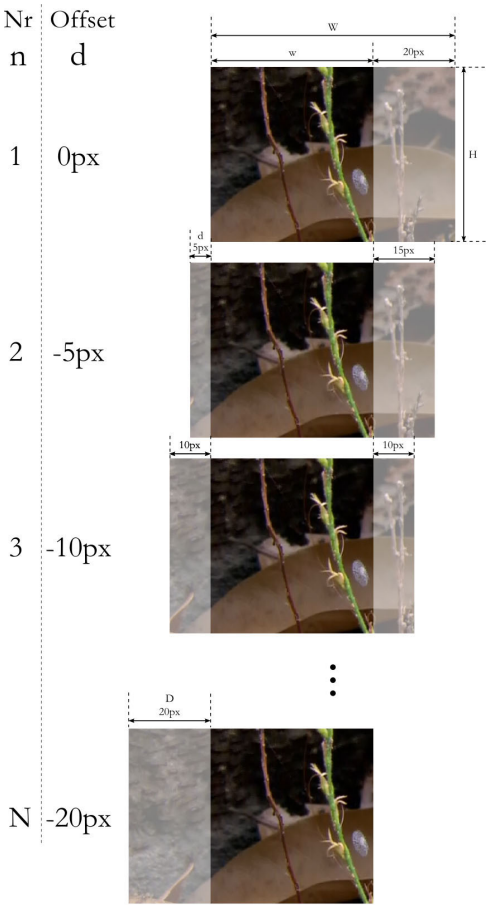


Figure 3: Example on lateral drift. The common area for all the images is the area which is not shaded. In real cases, there is often much smaller drift corrections than what this example shows.

4.1.1 Adjust template ROIs

To be able to drift correct images, template ROIs (region of interest) need to be drawn. The ROIs act as reference when the image is drifting from slice to slice in the image sequence. As we can see in figure 4, the program has taken the yellow marked area as a template ROI. Then, when applying the drift correction, the program will compare the first image of the sequence with all the other images one by one in chronological order in that yellow marked area. If the program sees that e.g. a quantum dot was moved one pixel to the left compared to the first slice, then it drift corrects by cutting off the rightmost column of pixels, and then corrects the drift by moving the whole image one pixel to the right.

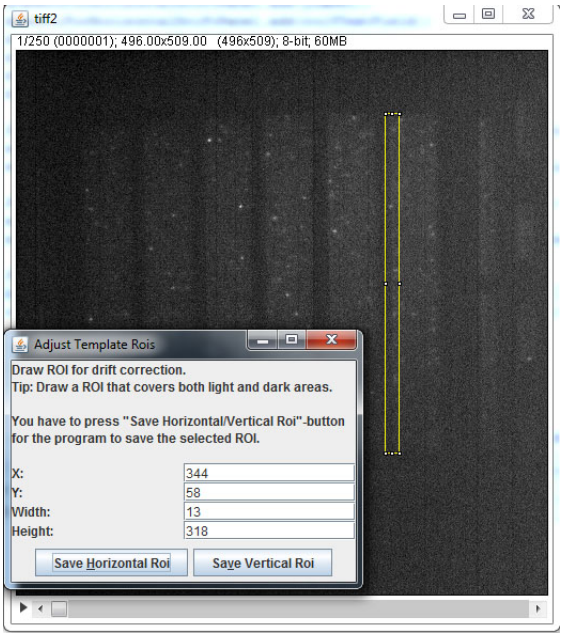


Figure 4: The yellow marked area is a template ROI for horizontal drifts. You can see its x- and y-coordinates, width and height in the figure.

4.1.2 Correlation coefficient

In [1, p. 434] the correlation coefficient is defined as follows:

$$C_L(r,s) = \frac{\sum_{(i,j)\in R}(I(r+i,s+j) - \overline{I}(r,s)) \cdot (R(i,j) - \overline{R})}{[\sum_{(i,j)\in R}(I(r+i,s+j) - \overline{I}_{r,s})^2]^{1/2} \cdot [\sum_{(i,j)\in R}(R(i,j) - \overline{R})^2]^{1/2}} \quad (1)$$

where I is the search image, R is the reference image, r the shifted horizontal position of R, s the shifted vertical position of R, and $(i,j) \in R$ all possible template coordinates $\{(i,j)\,|\,0 \leq i < w_R, 0 \leq j < h_R\}$. The mean values $\overline{I}_{r,s}$ and $\overline{R}$ are defined as

$$\overline{I}_{r,s} = \frac{1}{K} \cdot \sum_{(i,j)\in R} I(r+i,s+j) \quad \text{and} \quad \overline{R} = \frac{1}{K} \cdot \sum_{(i,j)\in R} R(i,j) \quad (2)$$

where $K = |R|$ is the number of pixels in the reference image R.

The algorithm for computing of the correlation coefficient is listed in algorithms 2–4.

Algorithm 2 Algorithm for computing reference image constants.

Require: Reference image R
  wI = width of I, hI = height of I
  wR = width of R, hR = height of R
  $K = wR \cdot hR$
  `sumR` = 0, `sumR2` = 0
  Get R's pixel data and save it in an array `pixelsR`
  for $i = 0$ to $i = wR - 1$ **do**
    for $j = 0$ to $j = hR - 1$ **do**
      `aR` = `pixelsR`$[j \cdot wR + i]$
      `sumR` = `sumR` + `aR`
      `sumR2` = `sumR2` + `aR`^{2}
    end for
    `meanR` = `sumR` / K
    `varR` = $\sqrt{\texttt{sumR2} - K \cdot \texttt{meanR}^2}$
  end for

Algorithm 3 Algorithm for `computeMatch` which returns the array of correlation coefficients.

Require: Search image I
  wI = width of I, hI = height of I
  wR = width of R, hR = height of R
  $w = wI - wR$, $h = hI - hR$
  Create a new FloatProcessor[2] C with width $w + 1$ and height $h + 1$.
  Get C's pixel data and save it in an array `pixelsC`
  for $r = 0$ to $r = w - 1$ **do**
    for $s = 0$ to $s = h - 1$ **do**
      d = `getMatchValue`(r, s)
      `pixelsC`$[s \cdot w + r] = d$
    end for
  end for
  Return C

Algorithm 4 Algorithm for `getMatchValue`.

Require: r and s from the function `computeMatch`
  `sumI` = 0, `sumI2` = 0, `sumIR` = 0
  Get I's pixel data and save it in an array `pixelsI`
  Get R's pixel data and save it in an array `pixelsR`
  for $i = 0$ to $i = wR - 1$ **do**
    for $j = 0$ to $j = hR - 1$ **do**
      `aI` = `pixelsI`$[(s + j) \cdot wI + r + i]$
      `aR` = `pixelsR`$[j \cdot wR + i]$
      `sumI` = `sumI` + `aI`
      `sumI2` = `sumI2` + `aI`^{2}
      `sumIR` = `sumIR` + `aI` $\cdot$ `aR`
    end for
  end for
  `meanI` = `sumI` / K
  `matchValue` = $\frac{\texttt{sumIR} - K \cdot \texttt{meanI} \cdot \texttt{meanR}}{\sqrt{\texttt{sumI2} - K \cdot \texttt{meanI}^2 \cdot \texttt{varR}}}$

4.2 Threshold

4.2.1 Set Mutliplier

Depending on how bright the image sequence is, the threshold need to be adjusted to filter properly. To adjust the threshold, there is a multiplier which multiplies the

threshold level with a chosen value. The multiplier can be adjusted by the user. The higher value, the higher threshold level.

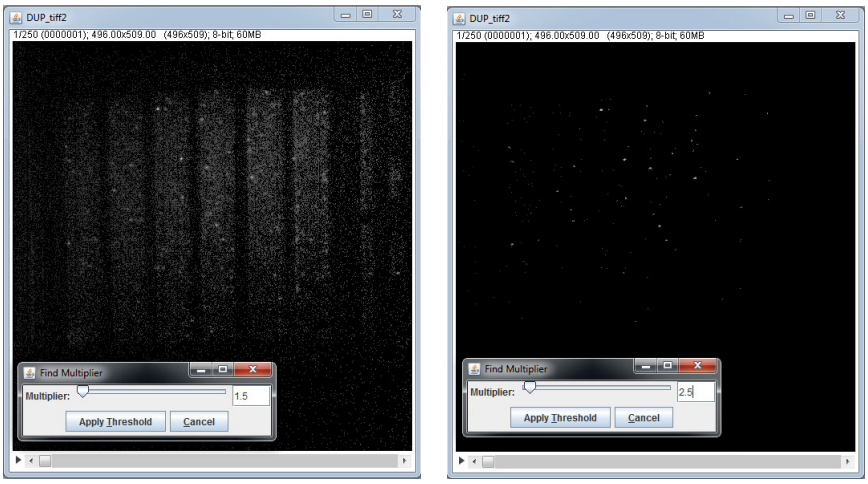
When choosing multiplier the following is important:

- if the multiplier value is too high, the program might not find any quantum dots.
- if the multiplier value is too low, the program might crash because of lack of memory which is caused by that the program has found too many quantum dots.

It might be difficult to determine what multiplier value should be chosen. Therefore, whenever the user loads an image (or an image sequence) into the program, an estimated multiplier value will be calculated for that specific image (or image sequence). The program uses equation (3) to calculate the estimated multiplier M.

$$M = 41.3638 - 10.16 \ln(\overline{m}) \tag{3}$$

where $\overline{m}$ is the mean value of the intensity of the image (or image sequence) which is calculated by a built-in ImageJ function. Since the formula is based on empirical results it only gives an approximate value of the multiplier. In figure 5(a) and 5(b) the program's multiplier adjustment is illustrated.



(a) (b)

Figure 5: (a) An image sequence filtered with multiplier value 1.5. (b) Same image sequence filtered with multiplier value 2.5.

4.2.2 Threshold map

Due to issues in the experimental equipment and setup, the recorded images do not in general have homogeneous illumination. In other words, the illumination might for instance be more intense on the right part of the image whereas the left part is less illuminated.

To compensate for this imbalance in illumination, the program therefore determines an illumination factor for each pixel and stores this as a threshold map. For the case of simplicity, the threshold map is based on a single image in the sequence. In the general case, the illumination may be different from image to image and one would then create a threshold map for each image. Algorithm 5 illustrates how the program creates the threshold map (from a single image).

Algorithm 5 Creation of a threshold map.

X = Image width, Y = Image height
Create a 2D array for the threshold map T
Compute and store the mean value of the intensities at the corner regions of size N^2 into $R_{topleft}$, $R_{topright}$, $R_{bottomleft}$, $R_{bottomright}$
$\Delta_x = \frac{R_{topright} - R_{topright}}{X}$
$\Delta_y = \frac{R_{bottomleft} - R_{topleft}}{Y}$
Compute the threshold map:
for $x = 0$ to $X - 1$ **do**
  for $y = 0$ to $Y - 1$ **do**
    T$(x, y) = R_{topLeft} + x \cdot \Delta_x + y \cdot \Delta_y$
  end for
end for
return Threshold map T

4.2.3 Applying the thresholding

After the program has created a threshold map, the thresholding of the image sequence can be done. To preserve the original image sequence, it is duplicated before starting the thresholding algorithm. The following is therefore operated on the duplicate image sequence.

The algorithm will go through every pixel in the image and compare its intensity to the threshold level, which is defined as the product of the (global) multiplier value and the intensity of the corresponding value in the threshold map. If the current image pixel value is below this threshold level, the pixel value is set to zero (0), i.e. black. Correspondingly, if the pixel value is equal or higher than the threshold level, the pixel value is unchanged. Hence, the algorithm produces a filtering of the image sequence which facilitates the following process of extracting quantum dots. Pseudocode for the thresholding process is provided in algorithm (6).

Algorithm 6 Algorithm for the threshold process.

Require: Threshold map T , multiplier M

Perform thresholding:
for each image in the sequence **do**
  Retreive the 2D pixel array I
  for each pixel value $p \in I$ **do**
    if current pixel value $p < (T \cdot M)$ **then**
      $p = 0$
    end if
  end for
end for

4.3 Extracting Quantum Dots

For extracting the quantum dots, we have used ImageJ's built-in function called "Particle Analyzer". In our algorithm, this function serves to detect the separated regions in the thresholded image and then compute statistical data from these regions. To evaluate the intensity of the quantum dots, we collect the integrated density using the "Particle Analyzer".

Pseudo-code for the quantum dot extraction process is listed in algorithm 7. The result of the process is illustrated in figure 6.

Unfortunately, the ImageJ built-in function `"Particle Analyzer"` sometimes misses to analyze quantum dots, or counts in quantum dots that don't look like quantum dots. Therefore, we have made a function which can manually add and remove quantum dots.

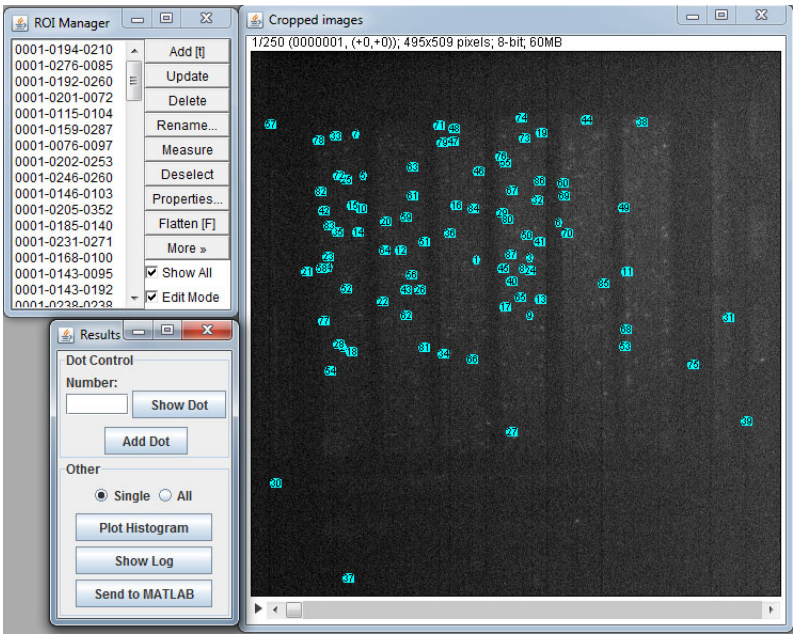


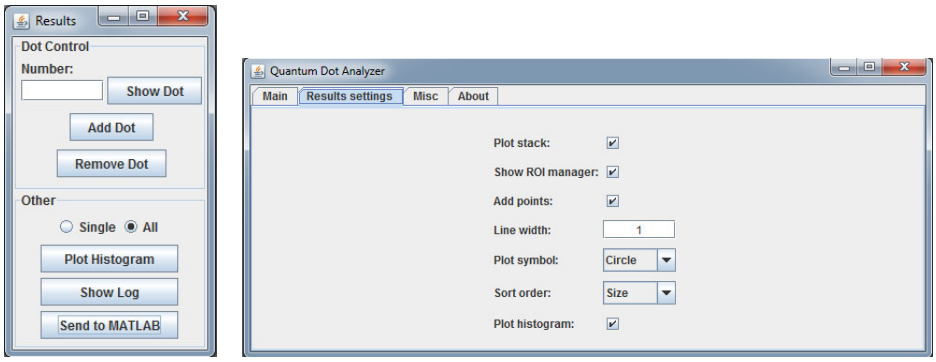
Figure 6: The blue numbered dots are the quantum dots which has been registred by the program.

Algorithm 7 Algorithm for the particle analysis.

Require: Original image sequence with a superposed thresholded layer I_T
  δ = Allowed deviation
  N = Number of images in the sequence
  for $n = 0$ to $N - 1$ **do**
    $I_T^{(n)}$ = slice n from the image sequence
    Analyze $I_T^{(n)}$ using the "Particle Analyzer" which produces a table with statistics of the identified particles/quantum dots, where each line $0 \leq p \leq P - 1$ represents one particle for the current image n. The statistical data include integrated density, which is used to determine intensity of the quantum dot.
    for $p = 1$ to $P - 1$ **do**
      Store the coordinates (x, y) and size (area) of the found particle/quantum dot p in x_p, y_p and A_p, respectively
      Find the closest quantum dot that has already been registered and store the coordinates and size in x, y and A
      Register new quantum dot in the database if the distance is larger than the allowed deviation δ, otherwise just add the statistics from the found particle p to the existing quantum dot:
      if $|(x_p - x, y_p - y)| > \delta$ **then**
        Create a new quantum dot in the database and copy the statistics from the found particle p for the current image n
        if $n > 0$ **then**
          Reanalyze the images $0 < n$ and add the corresponding statistics into the new quantum dot
        end if
      else
        Add the statistics to the existing quantum dot
        Update the max. size if necessary:
        if $A > A_{max}$ **then**
          $A_{max} = A$
        end if
      end if
    end for
    Clear the table of particles
  end for

4.4 Plotting the distrubution for on- and off-time

When the analyzing part is finished, the "Results" window shows up (figure 7(a)). Depending on what the user have chosen to show in the "Results settings"-tab (figure 7(b)), it might be more than just the "Results" window showing up.



(a) (b)

Figure 7: (a) The "Results" window. (b) The "Result settings"-tab window.

As we can see in figure 7(a), there is a "Send to MATLAB" button. If the user presses this button, the analyzed data will be sent to ImageJ's plugins folder and the program will attempt to start Matlab for post-processing. The script for post-processing is then able to generate additional statistics e.g. occurrence of on- and off-times.

4.5 Post-processing

By saving the analyzed data as for instance .txt files, we can post-process the data in Matlab. We have created a program in Matlab where you can plot intensity versus time graphs, histogram for the intensity, and occurence of on- and off-times. You can see the plots in figure 10, 11(a) and 11(b).

5 Results

5.1 Graphs of quantum dot light intensity

With time, different quantum dots may blink with different intensities. For figure 8, a sequence of 150 slides has been taken with 5 seconds interval. From the picture you can see that the quantum dot has two states. For about the first 600 seconds the quantum dot is in the off-state (around 1300 intensity a.u.), and after that it switches into the on-state (around 1600 intensity a.u.). During the on-state, at $t = 700, 850, 1000, 1050, 1100$ s the quantum dot switches into the off-state for some seconds.

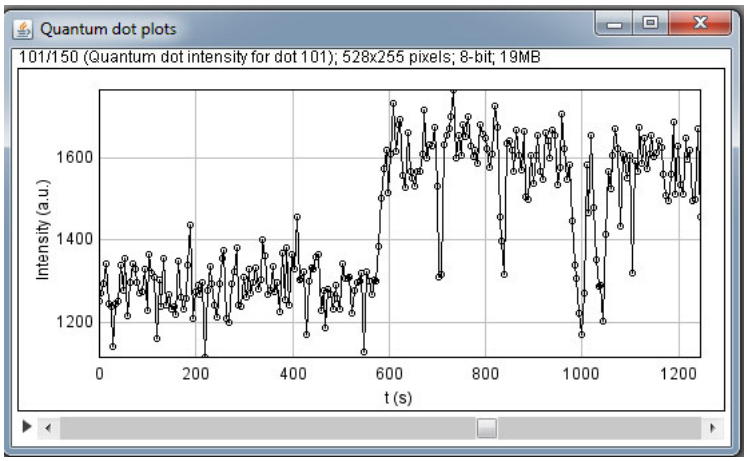


Figure 8: Intensity versus time graph for a quantum dot.

5.2 Log of data

Figure 9 shows a log of the analyzed quantum dots' data. It gives the user information about the quantum dot's number, position, max size and intensity. This window will show up if the user press the `"Show Log"`-button in the "Results"-window (see figure 7(a)).

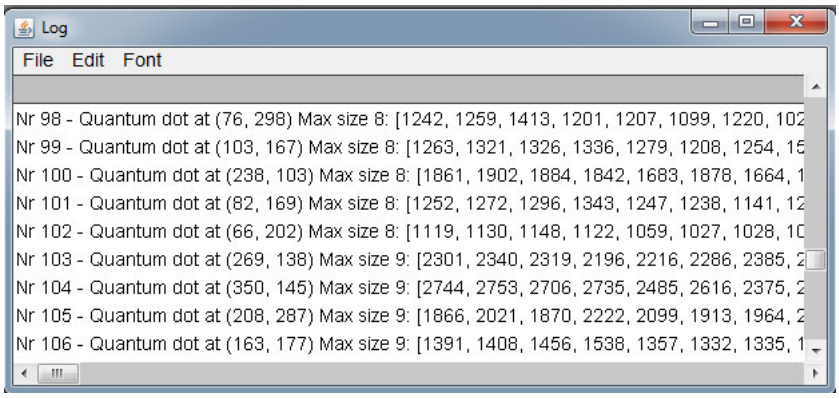


Figure 9: Log of data for the analyzed quantum dots.

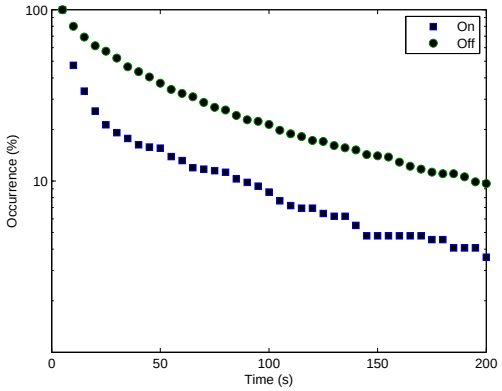
5.3 Graphs of on- and off-time

Figure 10 shows plots over the on- and off-time occurrencies for an image sequence. In this case we can see that the off-time occurs more often than the on-time. We

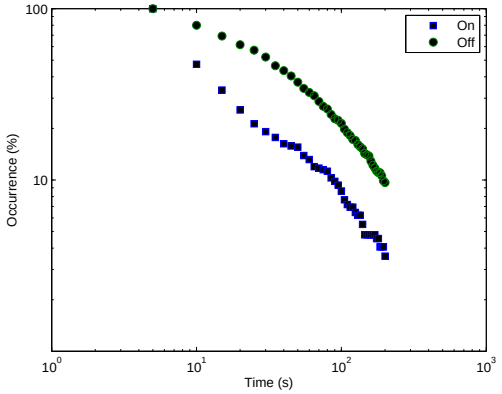
can see that the on- and the off-time is approximately linear in log-linear scale. This indicates the the quantum dots have roughly exponential dependency. In [2], the following switching model is used for the distribution of the on and off states:

$$p_{\text{on,off}}(t) \sim \exp(-\frac{t}{\tau_{\text{on,off}}}) \tag{4}$$

where τ_{on} and τ_{off} are average durations of on and off states correspondingly. When plotted on a logarithmic scale, equation (4) would yield linear graphs, which in most parts corresponds to the experimental results from the analysis – see figure 10.



(a) Log-linear scale



(b) Log-log scale

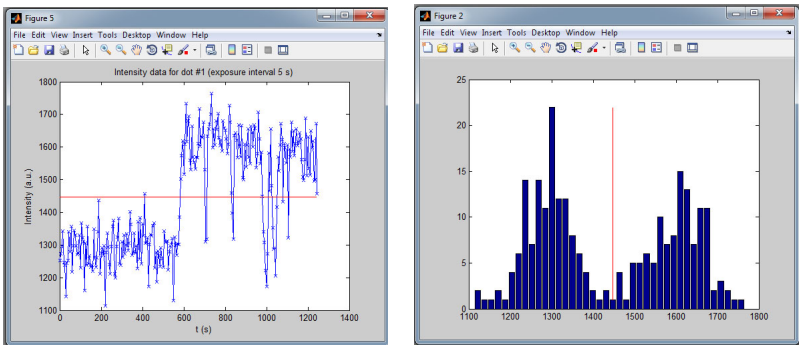
Figure 10: Plots over the on- and off-time for an image sequence, plotted using (a) logarithmic scale for the occurrence and (b) pure logarithmic scales.

5.4 Graph and histogram

Figure 11(b) shows a histogram in Matlab of the occurence of a quantum dot's intensity. The red line in the middle represents the boundary between on- and off-states. The red line is located at the middle point between the two highest peaks in the histogram. With this, we can see how long the quantum dot is in the on-state with respect to the off-state. For instance, at $t = 400$ in figure 11(a), the quantum dot is on for one second.

6 Discussion

During the development of the software, we have mostly worked with one specific image sequence. For this sequence, the software can distinguish most of the quantum



(a) (b)

Figure 11: (a) A graph showing a quantum dot's intesity over time. (b) A histogram showing the occurence of a quantum dot's intensity.

dots. When we try to use another image sequence, the software registers some quantum dots that do not look like quantum dots – in most cases noise – and fails to register some real quantum dots. This might depend on problems in the algorithms or that the contrast of the images are not high enough.

7 Conclusion

The Quantum Dot Analyzer program, which is based on Java and ImageJ, can analyze photoluminescense images of quantum dots and extract information such as apparent size, light intensity over time, on- and off-time occurence etc.

Even though the program might not be able to find all the quantum dots, or might include dots that are not true quantum dots, the user can manually add or remove quantum dots to compensate for this.

Sometimes there might be some shifting between the slices in a image sequence, which can disturb the analyzing. Therefore, we have created a function that corrects the shift.

Every image sequence has its own brightness, and sometimes the brightness can make it hard to distinguish the quantum dots. For this, the program has a function which is called multiplier, which acts like a threshold. With this you can adjust the brightness of the image sequence, and make it easier to distinguish the quantum dots. But the user has to be careful not to choose a multiplier value that makes the image sequence too dark, quantum dots with weak light intensity might be lost then.

The program has a Matlab feature where you can post-process the analyzed data of the quantum dots in Matlab. Here you can obtain graphs for the light over time, histogram of light intensity and on- and off-time occurence.

8 Recommendations

For the drift correction, as outlined in section 4.1, it is required that the position of the reference image is placed very accurately. In addition, this also required some trial-and-error to get a good result.

Optimally, the drift correction should be done without the need of manual input such as providing a reference image. This could be done by employing a method that evaluates each image and obtains a scalar value determining the drift in each direction. Due to the noise in the experimental data and different lighting conditions throughout the image, devising such an automatic algorithm is not trivial.

Another way is to track the centre of a single dot – or even multiple dots – through a time series. Due to the blinking behaviour of the quantum dots, the position can not be determined at all times. Therefore, if the tracked quantum dots

are "off" long periods of time, the measurement of the drift may not be accurate. The selection of the tracked dot(s) can be done by the user or by an automated process.

The post-processing was done in Matlab due to its built-in functionality and matrix handling. To provide a more complete package, most features of the post-processing, such as histogram generation, could be integrated into the main program.

9 References

References

[1] Wilhelm Burger and Mark James Burge. *Digital Image Processing: An Algorithmic Introduction using Java*. Number 978-1-84628-379-6. Springer Science+Business Media, LLC, first edition, 2008.

[2] Ilya Sychugov, Robert Juhasz, Jan Linnros and Jan Valenta. "Luminescence blinking of a Si quantum dot in a SiO_2 shell". (DOI: 10.1103/PhysRevB.71.115331), 2005.